

Enterprise Financial Risk Management Platform

Product Requirements Document

STAKEHOLDER REVIEW DRAFT | VERSION 1.0

Document ID	FRM-PRD-001
Product	FinFactory Enterprise Financial Risk Management (EFRM)
Prepared	2026-07-22
Coverage	Screening, AMC, Rule Engine, ML Engine, Admin Service, Admin Portal
Evidence posture	Verified current behavior plus explicitly labelled target scope

This document contains no passwords, tokens, secrets, PII, raw payloads, or real operational records.

Enterprise Financial Risk Management Platform

Product Requirements Document

Document ID: FRM-PRD-001 **Version:** 1.0 **Status:** Draft for stakeholder review **Prepared:** 2026-07-22 **Product:** FinFactory Enterprise Financial Risk Management (EFRM) **Scope:** Screening Service, AMC Service, Rule Engine Service, ML Engine Service, Admin Service, Admin Portal, shared data, security, workers, and operations

***Purpose.** This document separates verified current behavior from approved target scope. It is a product baseline, not a claim that every capability is active in every client database. Each requirement is classified as IMPLEMENTED, CONFIGURATION-DEPENDENT, PARTIAL, PLANNED, or OPEN.*

Document Control

Approval Record

Role	Name	Decision	Date
Product owner	To be nominated	Pending	-
Engineering owner	To be nominated	Pending	-
Security and operations owner	To be nominated	Pending	-
Compliance/business owner	To be nominated	Pending	-
Client/UAT approver	Per implementation	Pending	-

Change History

Version	Date	Author	Change
1.0	2026-07-22	Product and engineering working draft	Initial platform PRD based on repository, API, database, configuration, and portal review.

Evidence Boundary

The implementation review covered the six canonical repositories and the local PostgreSQL client databases Kanji_011, efrm_niyo, and efrm_uat_latest on the local development instance. No passwords, tokens, secrets, personal data, raw operational payloads, or real operational records are included in this PRD.

1. Executive Summary

FinFactory EFRM is a modular financial crime and fraud risk-management platform. It receives customer, transaction, device, screening, and adverse-media signals; evaluates them through deterministic rules and configurable scoring; optionally applies machine-learning analysis; creates operational alerts and investigation cases; maintains risk profiles; and gives authorized users controlled workflows for investigation, decision, evidence, audit, and compliance review.

The product is designed as a layered system:

- Detection:** Screening, adverse media, transaction rules, device rules, metrics, and ML inference identify signals.
- Decision:** Rules and configured policies translate signals into decisions, scores, priorities, and alert states.
- Risk:** RAMS ingests eligible events and applies configured domain impact, aggregation, decay, overrides, relationship propagation, and tiering.

4. **Investigation:** Admin Service groups alerts into cases, assigns work, tracks SLA, captures findings, captures a case outcome, initiates configured actions, and records evidence and audit history.
5. **Operations:** Admin Portal exposes authorized workflows, configuration, governance, monitoring, and review surfaces.

Client SOPs, enabled services, institution identifiers, risk-domain participation, screening sources, adverse-media availability, case decisions, action mappings, and integration endpoints are client-scoped policy. They are not universal product assumptions.

1.1 Product Position

EFRM is not only a rule engine and not only a case-management application. Its value is the controlled path from a source event to an explainable investigation decision:

TEXT DIAGRAM / REFERENCE BLOCK

```
Source signal
-> service-specific evaluation
-> alert/result and risk event
-> case correlation and investigation
-> human decision and evidence
-> configured operational action
-> audit, compliance, and risk-profile update
```

1.2 Current-State Qualification

The platform has strong foundations in deterministic rule processing, screening ingestion and evaluation, case decision/action orchestration, security capabilities, RAMS processing, and database-backed audit/evidence structures. UI maturity is uneven. Rule authoring, screening configuration, user/profile administration, SLA, and auto-assignment have stronger UI coverage. Case SOP configuration, RAMS policy configuration, Link Analysis configuration, selected shared catalogs, AMC child scoring configuration, and ML governance require further UI/API governance before they should be represented as fully self-service capabilities.

2. Product Purpose and Business Outcomes

2.1 Business Problem

Financial institutions need to identify suspicious activity across heterogeneous channels, explain why a signal was raised, coordinate investigation, apply institution-specific SOPs, preserve evidence, and demonstrate controlled decision-making to internal audit and regulators. Fixed lists and disconnected alert queues create gaps:

- one entity can appear across screening, adverse media, transaction, and device workflows;
- an alert severity is not the final investigator decision;
- client SOPs differ by institution and service;
- machine-generated risk signals and human outcomes must remain distinguishable;
- external action requests require delivery tracking without falsely claiming bank-side completion;
- evidence and audit records must be discoverable and tamper-resistant;
- configuration changes must be governed separately from runtime data.

2.2 Goals

Goal	Outcome
Detect	Identify suspicious patterns and watchlist/adverse-media matches across supported services.
Decide	Convert deterministic and analytical findings into explainable decisions and risk signals.
Investigate	Give authorized users one case context for related alerts, assignments, SLA, evidence, and history.

Goal	Outcome
Orchestrate	Apply client-configured case outcomes and initiate the actions required by the SOP.
Govern	Version, validate, authorize, and audit high-impact policies and access controls.
Explain	Preserve input, rule/model/configuration, score, decision, and evidence lineage.
Operate	Support retries, worker execution, observability, failure isolation, and environment-specific deployment.

2.3 Non-Goals

- replacing a bank core banking, card, payment-switch, CRM, recovery, or notification system;
- claiming business completion of a bank-side action without a confirmation contract;
- rewriting original machine-generated transaction, screening, device, or adverse-media result history after human review;
- providing an unrestricted client-facing SQL editor or raw secret editor;
- universal workflow-engine functionality, message-broker migration, compensation orchestration, or a visual SOP designer;
- treating ML mock/demo pages or placeholder reports as production capabilities;
- changing mature rule-authoring behavior without a specifically approved rule-engine change.

2.4 Success Measures

These are proposed measures and require product-owner baselines before becoming contractual SLAs:

Measure	Target definition
Traceability	Every production alert and case decision traces to its source record, actor, timestamp, and effective configuration where available.
Decision integrity	A case cannot close through a direct status update that bypasses the configured decision flow.
Tenant isolation	No API or query returns another institution's data or configuration.
Action reproducibility	Action execution preserves the endpoint, payload, header, retry, and timeout snapshot used for dispatch.
Operational clarity	SENT means the target accepted the request according to configured criteria; it does not mean bank-side business completion.
Evidence integrity	Evidence has ownership, hash/custody information, secure access, and discoverable events.
Configuration safety	High-impact changes have validation, authorization, audit, effective-state visibility, and a promotion path.

3. Personas and Responsibilities

Persona	Responsibilities	Main surfaces
Analyst/investigator	Review alerts, investigate entities, resolve alert findings, select case outcome, capture remarks/evidence.	Case Queue, Case Detail, Decision, Timeline/Evidence, Risk Profile
Supervisor	Approve decisions, review escalations, resolve operational exceptions, inspect SLA/evidence.	Assignments, Approval, Timeline/Evidence, SLA
Compliance officer	Review suspicious activity, STR/CTR indicators, rationale, evidence, and audit readiness.	Case Detail, Compliance Readiness, Audit
Case administrator	Manage case scoring, SLA, assignment, decisions, actions, mappings, and endpoints.	Case Configuration

Persona	Responsibilities	Main surfaces
Screening administrator	Manage sources, mappings, entity forms, fields, thresholds, bands, and exceptions.	Screening Configuration
Risk administrator	Manage RAMS versions, event admission, weights, impacts, decay, tiers, and overrides.	RAMS Policy Studio, Risk Profile
Rule administrator	Create and govern rules, metrics, groups, bindings, policies, and versions.	Rule Engine Configuration
ML administrator/governance reviewer	Manage features, models, training, champion/rollout state, governance, feedback, and explainability.	ML Configuration and Model Lifecycle
Security administrator	Manage profiles, capabilities, password/security policy, sessions, and service clients.	Administration and Security
Operations/SRE	Deploy services, operate workers, monitor health, manage retries, backups, and recovery.	Deployment and Operations tooling
Auditor	Review immutable audit, evidence custody, case chronology, and exports.	Timeline/Evidence and Technical Audit

4. Product Architecture

4.1 Logical Architecture

MERMAID DIAGRAM / REFERENCE BLOCK

```

graph LR
    S[Source systems and client channels] --> R[Rule Engine Service]
    S --> SC[Screening Service]
    S --> AM[AMC Service]
    R --> AS[Admin Service]
    SC --> AS
    AM --> AS
    R --> ML[ML Engine Service]
    SC --> RAMS[RAMS in Admin Service]
    AM --> RAMS
    ML --> RAMS
    AS --> RAMS
    AS --> LA[Link Analysis V2]
    AS --> W[Workers]
    AS --> DB[(PostgreSQL efrm schema)]
    SC --> DB
    AM --> DB
    ML --> DB
    R --> DB
    P[Admin Portal] --> AS
    P --> SC
    P --> AM
    P --> ML
    P --> R
  
```

4.2 Service Ownership

Component	Owns	Does not own
Screening Service	Watchlist sources, ingestion, screening, matching, screening results/alerts, screening configuration.	Transaction/device rules, bank-side actions, final case closure.
AMC Service	Adverse-media providers, retrieval/extraction/analysis, results/matches/alerts, adverse scoring configuration.	Screening ingestion, deterministic rules, final case closure.
Rule Engine Service	Source transformation, facts, rules, metrics, groups, Drools evaluation, decision policies, transaction/device results.	Human investigation, case SOP, bank-side execution.
ML Engine Service	Features, models, training, inference, anomaly score, explanation, feedback, workers.	Deterministic rules, case closure, final risk aggregation ownership.
Admin Service	Case lifecycle, alert mapping, assignment/SLA, case outcomes/actions, RAMS, Link Analysis, operational review, security/admin support.	Source-specific detection algorithms and bank-side completion.

Component	Owns	Does not own
Admin Portal	Authorized user experience and configuration/operation presentation.	System of record, policy enforcement by itself, secret storage, backend validation.

4.3 Shared Data Boundary

Services use a PostgreSQL efrm schema. A deployment may give an institution its own database or share infrastructure with strict institution_id isolation. PostgreSQL is the system of record for business configuration, runtime records, derived state, audit history, and worker state. Environment and secret configuration remain outside ordinary client CRUD.

5. Capability Catalogue

5.1 Transaction and Device Detection

The Rule Engine receives transaction and device inputs through API, GUI/manual, and bulk flows. It normalizes source payloads, resolves source-to-engine attributes, loads effective rule groups, resolves metrics/dependencies, executes Drools rules, applies score/decision policy, persists results and alerts, and can asynchronously create/correlate cases and submit eligible RAMS events.

Status: IMPLEMENTED with client/source configuration dependencies. Mandatory payload fields, source identifiers, timestamps, and event fields are contract-specific.

5.2 Screening and Watchlist Management

Screening supports watchlist source configuration and ingestion, source validation, column mapping, list-version lineage, entity/alias storage, dynamic entity forms, related entities, fuzzy matching, identifiers, thresholds, risk bands, whitelist/suppression, alerts, and operational review. It can integrate with AMC, RAMS, and cases when enabled.

Status: IMPLEMENTED for core runtime and ingestion; CONFIGURATION-DEPENDENT for sources, lists, form definitions, thresholds, and downstream integration.

5.3 Adverse Media / AMC

AMC accepts an adverse-media request, persists it, acknowledges asynchronous processing where used, resolves provider/runtime configuration, searches and retrieves content, extracts and analyzes it, computes decision/risk, persists results/matches, creates an alert, and optionally feeds RAMS and cases.

Status: IMPLEMENTED with CONFIGURATION-DEPENDENT provider availability. A transaction/device-only client may intentionally omit AMC and screening.

5.4 Rule Authoring and Governance

Rule administration provides rule identity, versioning, authoring/testing, required data, source/engine attributes, groups, source bindings, metrics, decision policy, upgrades, and activation/governance. The Rule Engine consumes effective versions and bindings at runtime.

Status: IMPLEMENTED and mature for existing rule paths. Unrelated feature work must not change this behavior.

5.5 ML Feature, Model, and Inference Lifecycle

ML supports feature sets and versions, feature definitions/metadata, model registry and versions, training jobs, artifacts, live/test/bulk inference, anomaly score/decision mapping, explanations/SHAP where available, feedback, and governance/rollout concepts.

Status: Runtime and core configuration are present. Some portal/API governance and lifecycle surfaces are PARTIAL or OPEN. Mock-backed pages must not be represented as live production operations.

5.6 Case Management

Case Management is the investigation container. Screening, adverse, transaction, and device alerts can be correlated to an existing open case by institution/entity context. `case_alert_mapping` uses source table plus source alert ID as identity so equal numeric IDs from different alert tables coexist.

TEXT DIAGRAM / REFERENCE BLOCK

```
source alert
-> case scoring and merge decision
-> case_master and case_alert_mapping
-> assignment and SLA
-> investigation
-> alert-level finding
-> case-level outcome
-> approval when configured
-> case closure
-> action queue and dispatch/audit
```

Status: IMPLEMENTED for case creation, mapping, assignment, SLA, alert-level decision, case-level outcome/action orchestration, evidence, audit, and timeline, subject to configuration and source integration state.

5.7 Outcome and Action Orchestration

Client outcomes are stored in `case_decision_master`. Reusable actions are stored in `case_action_master`; ordered mandatory/optional mappings are stored in `case_decision_action_mapping`; target endpoint/template/retry policy is stored in `integration_endpoint_config`.

Case closure validates the outcome and required fields, handles approval, preserves alert-level findings, stamps unresolved mappings, updates source alerts, persists case decision/status, queues actions, and records audit/events. V1 tracks delivery. SENT means the target accepted the request according to configured success criteria; it does not assert bank-side business completion.

5.8 RAMS / Risk Rating

RAMS consumes eligible `risk_event_log` events from transaction, device, screening, adverse, ML, relationship, and case-decision sources. It applies event admission, domain/service weights, decision impact, decay, tier, override, relationship propagation, aggregation, profile, factor, and history policy.

Case decisions use `source_type` CASE and `event_type` CASE_DECISION. Decision-impact configuration can increase, suppress, reduce, or override the human investigation signal. Original machine-generated events remain unchanged.

Status: Runtime IMPLEMENTED; policy administration UI is largely BACKEND API ONLY.

5.9 Link Analysis V2

Link Analysis builds configurable entity graphs from source attributes and dimensions, refreshes graph state through API/worker paths, stores nodes/edges/evidence/clusters/checkpoints, and presents case/global views. Configuration determines dimensions, source paths, relationship semantics, lookback/decay, limits, shared-count logic, masking, and refresh.

Status: Operational V2 visualization and refresh are present; configuration UI is DB/SQL ONLY and requires controlled administration before client self-service.

5.10 Evidence, Audit, Timeline, and Compliance

The unified case-history workspace separates:

View	Question answered	System of record
Timeline	What business events happened to this case, and in what order?	<code>case_events</code>
Evidence and custody	What files, versions, hashes, and custody events exist?	<code>case_evidence</code> and custody records

View	Question answered	System of record
Technical Audit	Who changed what, when, from where, with which old/new values?	audit_trail
Compliance Readiness	Is the case decision, approval, SLA, evidence, action, and reporting evidence sufficient?	Correlated case, audit, evidence, action, SLA, and decision data

These datasets remain separate even when presented in one workspace. Operations that never created an audit row cannot be reconstructed by aggregation.

6. End-to-End Functional Flows

6.1 Transaction and Device Intake to Alert

1. A client source submits a transaction/device payload using its contract and institution identifier.
2. Rule Engine validates mandatory fields and normalizes source attributes.
3. Source and engine definitions map payload values into canonical facts.
4. Active rule group/version is resolved for institution/source/channel.
5. Metrics and dependencies are resolved under configured limits.
6. Drools evaluates rules and emits signals/matches.
7. Decision policy and upgrades produce decision/risk output.
8. Request/master/result/alert/match records are persisted.
9. Asynchronous case/RAMS paths consume results when configured.
10. Admin Portal reads the alert, signal, case linkage, and risk context.

Core configuration: source_system_master, source_attribute_def, engine_attribute_def, engine_flink_map, rule_master, rule_version, rule_group tables, metric_definition, rule_metric_dependency, rule_decision_policy, rule_decision_upgrade, efrm_service_config.

6.2 Screening Ingestion and Runtime Screening

MERMAID DIAGRAM / REFERENCE BLOCK

```
sequenceDiagram
    participant Portal as Admin Portal
    participant SS as Screening Service
    participant DB as Screening DB
    participant RAMS as Admin Service RAMS
    participant Case as Admin Service Case
    Portal->>SS: Configure source and mappings
    SS->>DB: Store source configuration
    SS->>SS: Download, validate, parse, normalize
    SS->>DB: Store list version, entities, aliases
    Portal->>SS: Submit screening request
    SS->>DB: Store request and input
    SS->>DB: Retrieve eligible candidates
    SS->>SS: Match and score
    SS->>DB: Store result, matches, alert
    SS->>RAMS: Send eligible risk event
    SS->>Case: Create or link case if enabled
```

Candidate retrieval is grounded in real list_entity/alias data; scoring does not invent a watchlist match. List versions and mappings provide lineage.

6.3 AMC Request to Adverse Alert

1. Caller submits adverse request with entity context and institution.
2. AMC persists adverse_request and acknowledges asynchronous processing where used.
3. Runtime config resolves provider order, search, limits, timeouts, retries, and safe overlays.

4. Providers retrieve/search content.
 5. Extraction and analysis create category, severity, source, country, recency, and confidence signals.
 6. Configured weights and bands determine adverse score/risk.
 7. adverse_result, adverse_match, and adverse_alert are persisted; request status changes.
 8. Eligible results feed RAMS and case management.
 9. Portal presents adverse details, matches, case state, and risk context.
- Provider failure is not a confirmed adverse result; processing, result, and alert state remain distinct.

6.4 ML Configuration, Training, and Inference

MERMAID DIAGRAM / REFERENCE BLOCK

```

flowchart TD
    F[Feature set] --> FV[Feature set version]
    FV --> M[Model registry]
    M --> MV[Model version]
    MV --> G[Governance and rollout]
    G --> T[Training job]
    T --> A[Artifact and metrics]
    A --> I[Inference]
    I --> X[Feature engineering]
    X --> S[Score and decision]
    S --> E[Explanation]
    E --> R[Inference result and feedback]
    R --> T
  
```

The API creates or validates jobs; workers process training and bulk inference. API startup alone does not complete queued jobs. Model, feature, artifact, score, and governance lineage must be preserved.

6.5 Case Creation and Merge

1. A source alert/result arrives at Admin Service.
2. Source identity, entity identity, alert type, source table, priority/risk, and institution are derived.
3. Case scoring applies policy, alert category, count boost, correlation, and critical override.
4. Open-case lookup is institution/entity scoped.
5. An existing case is reused or a new case_master row is created.
6. case_alert_mapping is inserted using case ID, source alert ID, alert type, and alert source table.
7. Source alert status becomes CASE_CREATED when appropriate.
8. Assignment/SLA state is created or refreshed.
9. Case event/audit lineage is recorded.

6.6 Investigation, Decision, and Closure

Alert-level decisions are mapping findings. They validate context and required fields, persist mapping metadata, mirror legacy disposition values, close only the selected mapping/source alert when applicable, and create an event/audit record. They do not queue case-level actions or close the parent case.

Final case closure is the official business outcome:

TEXT DIAGRAM / REFERENCE BLOCK

```

POST /cases/{case_id}/close
-> validate state, outcome, required fields, and approval
-> preserve mappings already resolved
-> stamp unresolved mappings with final outcome
-> update corresponding source alerts to CLOSED
-> save case decision and case status
-> queue configured actions
-> emit case/audit events
-> commit transaction
  
```

V1 closes the RiskBrains case after the decision is accepted. Action dispatch remains separately tracked and does not claim bank-side business completion.

6.7 Action Dispatch

1. Outcome mappings resolve actions in sequence.
2. Endpoint configuration is snapshotted into the queue row.
3. Idempotency key is generated from case, decision, action, and sequence identity.
4. Internal or REST/webhook actions are queued.
5. The dispatch worker claims pending/retryable work.
6. The worker sends the stored request snapshot.
7. HTTP 2xx or configured success codes produce SENT; HTTP 202 may also set business_status to AWAITING_CONFIRMATION.
8. Failures create retry metadata and execution logs.
9. Exhaustion/manual resolution remains auditable.

6.8 RAMS Processing

MERMAID DIAGRAM / REFERENCE BLOCK

```
sequenceDiagram
    participant Source as Detection or case decision
    participant Ingest as Risk ingestion
    participant Config as RAMS configuration
    participant Agg as Aggregation
    participant DB as Risk tables
    Source->>Ingest: Eligible event
    Ingest->>Config: Resolve admission, domain, weight, impact
    Config-->>Ingest: Effective policy
    Ingest->>DB: Persist risk_event_log
    Ingest->>Agg: Process contributing event
    Agg->>Config: Resolve decay, tier, override, relationship
    Agg->>DB: Update aggregate/profile/factors/history
```

6.9 Link Analysis Refresh and Visualization

1. Dimensions/views identify source columns or JSON paths, node types, relationships, masking, lookback, confidence, and limits.
 2. Refresh API/worker selects source data and computes nodes/edges.
 3. Graph nodes, edges, evidence, clusters, and checkpoints are stored.
 4. V2 endpoints expose graph and cluster state.
 5. Portal loads global or case-specific graph data.
- Checkpoint state is worker state, not client-editable configuration.

7. Functional Requirements

7.1 Status Model

Status	Meaning
IMPLEMENTED	Verified in code/API/data model.
CONFIGURATION-DEPENDENT	Runtime exists but needs active client setup or source data.
PARTIAL	Some lifecycle or UI/API behavior exists, but the full contract is incomplete.
PLANNED	Approved target not verified as current.
OPEN	Requires product, security, compliance, or client decision.

7.2 Platform and Security

ID	Requirement	Status	Acceptance
FRM-PLT-001	Isolate institution-scoped data and configuration.	IMPLEMENTED	Cross-institution reads/writes are denied.
FRM-PLT-002	Resolve profile, menu, and capability access before protected operations.	IMPLEMENTED	Unauthorized endpoint calls fail even if a menu is hidden or visible.
FRM-PLT-003	Support password, session, token, security-check, and service-client controls.	CONFIGURATION-DEPENDENT	Secrets never return in plaintext.
FRM-PLT-004	Distinguish user, API-key, and machine-client authentication.	IMPLEMENTED	Contract-specific auth is enforced.
FRM-PLT-005	Keep audit/security history append-only for normal workflows.	IMPLEMENTED	No normal edit/delete path exists.
FRM-PLT-006	Govern high-impact security changes with authorization and audit.	PARTIAL	Profiles/security checks covered; endpoint maps/service clients need governed workflow.

7.3 Screening

ID	Requirement	Status	Acceptance
FRM-SCR-001	Ingest configured watchlist sources with source/version lineage.	IMPLEMENTED	Source, mapping, version, entity, and alias lineage is queryable.
FRM-SCR-002	Retrieve candidates from real configured list data.	IMPLEMENTED	No candidate means no fabricated match.
FRM-SCR-003	Apply configured field/group/identifier/threshold/band policy.	CONFIGURATION-DEPENDENT	Effective policy drives the result.
FRM-SCR-004	Support dynamic entity fields and related entities.	IMPLEMENTED	Active forms drive validation and accepted payload.
FRM-SCR-005	Protect screening alert state/decision updates.	IMPLEMENTED	Scope, validation, and audit apply.
FRM-SCR-006	Integrate with AMC/RAMS/cases only when enabled.	CONFIGURATION-DEPENDENT	Disabled services do not create downstream rows.
FRM-SCR-007	Make whitelist/exception handling attributable and time-bounded.	PARTIAL	Data model exists; governed UI/approval/expiry remains required.

7.4 AMC

ID	Requirement	Status	Acceptance
FRM-AMC-001	Accept and persist an adverse-media request before async work.	IMPLEMENTED	Request ID and state are available after acceptance.
FRM-AMC-002	Use configured provider/search/extraction/analysis policy.	CONFIGURATION-DEPENDENT	Effective source is identifiable and secrets masked.
FRM-AMC-003	Keep request, result, match, and alert state separate.	IMPLEMENTED	Provider failure is not a result decision.
FRM-AMC-004	Explain adverse score by configured factors.	IMPLEMENTED	Review can identify category/severity/source/country/recency effects.
FRM-AMC-005	Feed RAMS/cases only under client policy.	CONFIGURATION-DEPENDENT	Transaction/device-only profiles can omit AMC.

7.5 Rule Engine

ID	Requirement	Status	Acceptance
FRM-RUL-001	Transform source payload into canonical facts.	IMPLEMENTED	Mappings are applied consistently.
FRM-RUL-002	Resolve active rule groups and versions by context.	IMPLEMENTED	Effective binding and activation are respected.
FRM-RUL-003	Resolve metrics/dependencies within safety limits.	CONFIGURATION-DEPENDENT	Invalid dependencies and limit breaches fail clearly.
FRM-RUL-004	Execute Drools rules and apply decision policies/upgrades.	IMPLEMENTED	Signal, score, decision, rule, and version are persisted.
FRM-RUL-005	Preserve source identifiers and request lineage.	IMPLEMENTED	Source, internal, and result identities remain distinguishable.
FRM-RUL-006	Isolate slow downstream case/RAMS work from unrelated admin responses.	PARTIAL	Async/timeout isolation is verified per deployment.
FRM-RUL-007	Preserve rule version/activation/binding audit.	IMPLEMENTED	Actor and effective version are available.

7.6 ML

ID	Requirement	Status	Acceptance
FRM-ML-001	Define feature sets and versions before model use.	IMPLEMENTED	Active model references valid feature version.
FRM-ML-002	Preserve feature source/type/transformation/default semantics.	IMPLEMENTED	Vector construction is reproducible.
FRM-ML-003	Preserve model artifact/version/training/metrics/lifecycle metadata.	PARTIAL	Promotion requires complete lineage.
FRM-ML-004	Process training and bulk inference through durable workers.	IMPLEMENTED	Job state survives API restart.
FRM-ML-005	Resolve approved active/champion model and feature versions.	IMPLEMENTED	Inference identifies versions used.
FRM-ML-006	Govern model rollback and score blending.	PARTIAL	Auth/UI/contract gaps must close before self-service governance.
FRM-ML-007	Feed downstream RAMS/cases through explicit ML output contracts.	CONFIGURATION-DEPENDENT	Source, score, and provenance are preserved.

7.7 Case and Actions

ID	Requirement	Status	Acceptance
FRM-CASE-001	Create or merge a case from eligible source alerts.	IMPLEMENTED	Merge is institution/entity scoped and atomic.
FRM-CASE-002	Support mixed screening/adverse/transaction/device alert mappings.	IMPLEMENTED	Source table is part of mapping identity.
FRM-CASE-003	Apply assignment and SLA according to configured context.	CONFIGURATION-DEPENDENT	Assignee, SLA, escalation, and no-match state are visible.
FRM-CASE-004	Resolve alert-level findings without closing parent case or queuing case actions.	IMPLEMENTED	Only selected mappings/source alerts change.
FRM-CASE-005	Require configured case outcome and mandatory fields for close.	IMPLEMENTED	Direct status bypass is rejected.
FRM-CASE-006	Preserve resolved mappings and	IMPLEMENTED	Existing mapping decisions remain

ID	Requirement	Status	Acceptance
	stamp only unresolved mappings at final close.		unchanged.
FRM-CASE-007	Initiate actions and track delivery separately from case closure.	IMPLEMENTED	Queue/log snapshots and idempotency exist.
FRM-CASE-008	Retry/manual-resolve actions idempotently and audibly.	IMPLEMENTED	Duplicate dispatch is prevented and status is traceable.
FRM-CASE-009	Feed case decisions into RAMS as CASE/CASE_DECISION.	IMPLEMENTED	Decision impact changes risk without rewriting original events.
FRM-CASE-010	Protect evidence ownership, preview/download, hash, and custody.	PARTIAL	End-to-end secure deployment verification required.
FRM-CASE-011	Separate timeline, evidence, technical audit, and compliance views.	IMPLEMENTED	Each view uses its intended system of record.

7.8 RAMS

ID	Requirement	Status	Acceptance
FRM-RAM-001	Admit only configured source/event pairs.	IMPLEMENTED	Unsupported events do not change profile.
FRM-RAM-002	Apply domain/service weights and decision impacts.	CONFIGURATION-DEPENDENT	Effective policy is identifiable.
FRM-RAM-003	Maintain event, aggregate, profile, factor, and history lineage.	IMPLEMENTED	Profile changes can be explained.
FRM-RAM-004	Apply decay, tier, override, and relationship policy.	IMPLEMENTED	Precedence and result are documented.
FRM-RAM-005	Suppress/reduce false-positive case signals without deleting original events.	IMPLEMENTED	Original events remain unchanged.
FRM-RAM-006	Version and validate RAMS policy before activation.	PARTIAL	Backend exists; governed policy UI remains target scope.

7.9 Link Analysis

ID	Requirement	Status	Acceptance
FRM-LNK-001	Derive graph from configured dimensions/source fields.	IMPLEMENTED	Graph output is source-traceable.
FRM-LNK-002	Refresh graph through observable worker/API state.	IMPLEMENTED	Refresh status is durable and retryable.
FRM-LNK-003	Enforce graph node/edge/lookback/limit safety controls.	CONFIGURATION-DEPENDENT	Graph respects effective view policy.
FRM-LNK-004	Link clusters/evidence to source entity/case context.	IMPLEMENTED	Cluster/node/edge lineage is available.
FRM-LNK-005	Govern graph dimension/view configuration.	PLANNED	UI validates fields, masking, relationship, risk, and limits.

7.10 Portal and Operations

ID	Requirement	Status	Acceptance
FRM-UI-001	Authenticate and establish a backend-compatible session/token.	IMPLEMENTED	Invalid auth fails safely and no protected data loads.
FRM-UI-002	Resolve menu visibility from	IMPLEMENTED	UI visibility cannot bypass API

ID	Requirement	Status	Acceptance
	profile/capability mapping.		authorization.
FRM-UI-003	Show institution scope and validation state on configuration pages.	PARTIAL	High-impact workspaces add scope/version/effective-state context.
FRM-UI-004	Distinguish runtime data from configuration.	IMPLEMENTED	Queues/results are not generic configuration CRUD.
FRM-UI-005	Export authorized filtered data safely.	PARTIAL	Case audit CSV/Excel and evidence exports are verified per surface.
FRM-OPS-001	Run workers independently and restart safely.	IMPLEMENTED	Job/checkpoint state is durable.
FRM-OPS-002	Keep secrets and infrastructure policy deployment-managed.	IMPLEMENTED	UI exposes sanitized state only.

8. API and Integration Contracts

Every consumed API contract must specify authentication, institution scope, actor, request identity, idempotency, required/deprecated fields, sync/async semantics, status lifecycle, timeout/retry limits, validation errors, audit behavior, and ownership/retention.

Integration	Producer	Consumer	Main contract concern
Transaction/device intake	Client source or portal test flow	Rule Engine	Source IDs, payload mapping, mode, timestamps, idempotency.
Rule results	Rule Engine	Admin Service/RAMS/case flow	Result identity, decision, score, signal, source table, event timestamp.
Screening request/result	Caller/portal	Screening Service	Dynamic payload, identifiers, list policy, status, alert creation.
Screening downstream	Screening Service	AMC/RAMS/Admin Service	Optional enablement and no false downstream rows.
AMC request/result	Caller/Screening	AMC Service	Async request, provider failure, result/match/alert separation.
ML inference	Portal/client/service	ML Engine	Feature/model version, input contract, score/explanation.
Risk event	Detection/case services	Admin Service RAMS	source_type, event_type, entity, score, decision, reference lineage, duplicate protection.
Case action	Admin Service	Bank/external endpoint	Snapshot, template variables, auth reference, timeout/retry, idempotency, success status.
Link graph	Admin Service worker/API	Admin Portal	Refresh state, filters, source lineage, graph limits.
Audit/timeline	Admin Service	Portal/auditor	Case query, separation, sanitization, export format.

8.1 Action Status Semantics

- QUEUED: persisted for dispatch.
- PROCESSING: claimed by worker.
- SENT: target accepted request according to configured success criteria.
- FAILED: attempt did not meet success criteria.
- RETRY: retryable failure scheduled.
- RETRY_EXHAUSTED: retries consumed.
- MANUALLY_RESOLVED: approved operational resolution recorded.
- business_status=AWAITING_CONFIRMATION: optional business confirmation is not known.

9. Database and Configuration Model

9.1 Table Classes

Class	Examples	Lifecycle
Configuration/governance	Rule, screening, case SOP, RAMS, graph, profile/capability catalogs	Version, validate, authorize, activate, retire.
Runtime/result	Requests, results, alerts, matches, cases, mappings, inference	Lifecycle updates with source lineage.
Derived/aggregate	Risk profile/aggregate/factors, graph clusters, calculated metrics	Rebuild/recalculate; do not edit as policy.
Audit/history	audit_trail, case_events, security history, governance history	Append-only or controlled immutable history.
Worker/queue	Action queue/log, ML jobs, graph checkpoints, bulk jobs	Durable, retryable, operationally observable.
Legacy/uncertain	Older risk/rule/result tables and duplicate prototypes	REVIEW REQUIRED before migration or UI.

9.2 Configuration Inventory

The complete table-by-table inventory is maintained in PRODUCT_CONFIGURATION_UI_COVERAGE_AUDIT.md and PRODUCT_CONFIGURATION_UI_COVERAGE_DETAILED_AUDIT.md. It covers 94 configuration/governance tables across platform/security, case management, RAMS, Link Analysis, Rule Engine, Screening, AMC, and ML.

High-priority gaps are Case SOP, RAMS policy, Link Analysis policy, Case Scoring, service enablement/risk-domain mapping, whitelist exceptions, rule metric catalog, selected AMC scoring editors, and ML governance/blending.

9.3 Configuration Safety

High-impact configuration must be administered as a domain policy workspace, not arbitrary CRUD. Every workspace should provide institution scope, draft/effective version, validation, overlap/dependency checks, approval, activation, diff, mandatory change reason, audit event, promotion bundle, and safe retirement/rollback.

Secrets and infrastructure settings remain deployment-managed. Runtime rows, derived rows, queue state, checkpoints, and audit history are not client configuration.

10. State Machines

10.1 Screening and AMC

TEXT DIAGRAM / REFERENCE BLOCK

```

SCREENING: RECEIVED -> VALIDATING -> PROCESSING -> COMPLETED
              \-> FAILED
AMC:          ACCEPTED -> SEARCHING -> ANALYZING -> COMPLETED
              \-> FAILED/NO_RESULT

```

Alert state is separate from request processing state.

10.2 Rule and ML

TEXT DIAGRAM / REFERENCE BLOCK

```

RULE_VERSION: DRAFT -> VALIDATED -> APPROVED -> ACTIVE -> RETIRED
ML_VERSION:   DRAFT -> TRAINING -> TRAINED -> GOVERNANCE_REVIEW -> APPROVED
              -> SHADOW/CANARY/REALTIME -> ROLLED_BACK/RETIRED

```

Actual transition labels vary by service; effective state must remain visible and invalid transitions must fail.

10.3 Case and Action

TEXT DIAGRAM / REFERENCE BLOCK

```

CASE: OPEN -> IN_PROGRESS -> DECISION_SUBMITTED
      -> PENDING_APPROVAL -> ACTION_PENDING -> ACTION_DISPATCHED -> CLOSED
      \-> ACTION_FAILED

```

```

ALERT_MAPPING: OPEN/CASE_CREATED -> CLOSED
ACTION:        PENDING -> PROCESSING -> SENT
               \-> RETRY -> PROCESSING
               \-> FAILED/RETRY_EXHAUSTED/MANUALLY_RESOLVED

```

10.4 Evidence, Risk, and Graph

TEXT DIAGRAM / REFERENCE BLOCK

```

EVIDENCE: UPLOADED -> VERIFIED -> VIEWED/DOWNLOADED -> VERSIONED/RETAINED
RISK:      RECEIVED -> ADMITTED -> PROCESSED -> AGGREGATED -> PROFILED/HISTORIED
GRAPH:     REQUESTED -> RUNNING -> COMPLETED
           \-> FAILED -> RETRYABLE

```

11. Security, Privacy, and Compliance

11.1 Authorization

Authorization is layered:

1. Authentication establishes user or machine identity.
2. Institution context limits data/configuration scope.
3. Profile/menu visibility controls navigation.
4. Capability/endpoint maps protect APIs.
5. Service validation controls business transitions.
6. Audit records preserve actor and operation context.

Portal visibility is not a security boundary by itself.

11.2 Sensitive Data

Customer identifiers, transaction data, source payloads, watchlist data, provider content, evidence files, model artifacts, API credentials, signing keys, security data, and action payloads require masking, access control, retention, and secure logging.

11.3 Audit and Custody

Audit and custody records are append-only from normal product workflows. Corrections are new events, not silent edits. Direct database access is a restricted operational control, not a normal product function.

11.4 Compliance Boundary

The platform supplies evidence/audit/readiness support. It does not replace an institution's regulatory filing approval or submission process. STR/CTR recommendations/flags must be distinguished from filed reports.

12. Non-Functional Requirements

Area	Requirement
Performance	Enforce pagination, timeout, payload, query, and batch limits; move expensive work to asynchronous state.
Availability	Slow/failing downstream integrations must not take down unrelated admin read paths.
Scalability	Stateless APIs scale horizontally; workers scale by queue with idempotent claims.
Resilience	Use bounded retry/backoff; validation/configuration errors fail fast.

Area	Requirement
Observability	Correlation/request identity, structured statuses, metrics, traces, and secret-free logs.
Data integrity	Atomic case/source changes and idempotent/preflighted migrations.
Backup/recovery	Tested backup/restore for client databases, evidence, and model artifacts.
Retention	Policy-based retention and legal hold for audit/evidence.
Deployment	Environment-specific secrets/config and controlled schema migration order.
Compatibility	Legacy fields remain where required; deprecation is explicit.
Accessibility	Keyboard, responsive, clear error, and non-color-only status behavior.

13. Client Implementation Profiles

A client profile documents institution ID/database model, enabled services, entity identity/enrichment, screening/AMC availability, source payloads, rule/metric policy, RAMS domains/weights/tiers/impacts, graph dimensions/limits, case SOP/priority/SLA/assignment/decision/action/endpoint policy, retention/reporting, portal capabilities, and deployment/worker topology.

Profile	Typical scope	Deliberate variation
Transaction/device-only	Rule Engine, RAMS, Case, Link Analysis	No screening/AMC; entity may be created from transaction payload.
Screening-enabled	Screening, optional AMC, RAMS, Case	Dynamic entities, list sources, screening policy, optional adverse-media path.
Full FRM	All detection/investigation services	All downstream integrations are client/SOP controlled.

The profile is a deployment description, not a claim that every client has every feature active.

14. UAT Acceptance and Release Gates

Every UAT scenario identifies actor/profile/institution, source payload/precondition, effective configuration version, UI/API steps, expected state transition, database lineage, audit/event/action behavior, negative path, and evidence.

Review gates:

- Engineering:** code, API, schema, service, worker, and regression tests.
- Security/operations:** authentication, capability, tenant isolation, secrets, timeout/retry, observability, backup, deployment.
- Product/compliance:** business workflow, decision semantics, evidence/audit, regulatory wording.
- Stakeholder/UAT:** client profile, seed/config bundle, test cases, defect disposition, go-live acceptance.

Release is blocked if tenant scope/auth is missing, source lineage is lost, case close bypasses decision, source alert/case mapping can diverge, action retries can duplicate, model lineage is incomplete, evidence access bypasses ownership, required client service has no configuration, mock data is presented as live, or schema/config drift is unresolved.

15. Risks, Dependencies, and Open Decisions

Risk	Impact	Mitigation
Client configuration drift	UAT mismatch or wrong behavior	Versioned client bundles, schema checks, diff review, institution-aware promotion.

Risk	Impact	Mitigation
DB/environment split source of truth	Unexpected scoring/provider/rule behavior	Effective-config view and explicit source precedence.
External action ambiguity	False assumption bank completed operation	Separate delivery/business status and precise UI wording.
Missing source identity	Duplicate/untraceable events	Required source IDs, idempotency, contract tests.
Downstream slowness	Admin availability degradation	Async workers, timeout isolation, retries/circuit controls.
UI/API gap	Operator believes a change was saved	Contract tests, safe disabled controls, explicit status.
Mock surface	Stakeholder overestimates readiness	Production-backed classification and release gate.
Legacy ambiguity	Unsafe migration or duplicate behavior	REVIEW REQUIRED inventory and runtime-reference analysis.

Dependencies include approved SOPs, stable source contracts, target DB migrations/seed bundles, secret/deployment profiles, worker runtime, client endpoint contracts, ML artifacts/governance, and screening/AMC data licensing.

Open decisions include self-service Case/RAMS/Link configuration, signed bundle promotion, bank callbacks, ML governance/blending, performance/retention targets, legacy table disposition, and regulatory report/print scope.

16. Roadmap

Phase 0 - Correctness and Contract Integrity

- Repair unsupported Reference Data and Enum UI/API operations.
- Align ML feature/model/governance contracts and authentication.
- Classify/hide mock or placeholder production-looking surfaces.
- Formalize client configuration baselines and schema-diff checks.

Phase 1 - High-Value Governance

- Case SOP Decision/Action/Endpoint Studio.
- RAMS Policy Studio with simulation and effective versions.
- Case Scoring configuration workspace.
- Restricted service enablement/risk-domain mapping.

Phase 2 - Graph, Exceptions, and Rule Configuration

- Link Analysis Dimension/View/Refresh studio.
- Screening whitelist/exception workflow.
- Metric Catalog and engine/Flink mapping.
- Complete AMC child scoring editors.

Phase 3 - Governance Maturity

- ML governance and score-blending lifecycle.
- Endpoint-map diagnostics and service-client rotation.
- Signed configuration bundle export/import/promotion.
- Legacy table retirement after runtime-use confirmation.

17. Glossary

Term	Meaning
AMC	Adverse Media Checking service.

Term	Meaning
Alert	Operational signal requiring review; not itself the final business outcome.
Case	Investigation container grouping one or more source alerts.
Decision/outcome	Human/business conclusion selected during alert or case handling.
Disposition code	Legacy compatibility field retained where required.
EFRM	Enterprise Financial Risk Management platform.
Institution scope	Tenant/client boundary for data and configuration.
RAMS	Risk Assessment and Monitoring System risk-rating capability.
SENT	Target accepted request according to configured criteria; not bank-side completion.
SOP	Client standard operating procedure encoded as decisions, fields, approvals, and actions.

18. Source References

- Admin Service: docs/ADMINSERVICE_BUSINESS_LOGIC_OVERVIEW.md, docs/CASE_MANAGEMENT.md, docs/RISK_RATING.md, docs/RISK_RATING_API.md, docs/PRODUCT_CONFIGURATION_UI_COVERAGE_AUDIT.md, app/api/endpoints/case_api.py, app/services/case_management_service.py, app/services/case_decision_service.py, app/services/risk_processing_service.py, app/services/link_analysis_v2_service.py.
- Screening Service: AGENTS.md, docs/screening_implementation.md, docs/SCREENING_CONFIG_IMPLEMENTATION.md, docs/screening_data_source.md, app/api/endpoints/screening.py, app/api/endpoints/source_config.py, app/services/ingestion_service.py, app/services/fuzzy_screening_service.py, app/workers/ingestion_worker.py.
- AMC Service: AGENTS.md, adverse-media API/config routes, runtime-config/provider/analyzer services, and adverse ORM models.
- Rule Engine Service: Spring controllers/services, Drools resources, application.yml, rule/metric/entity repositories, and migrations.
- ML Engine Service: AGENTS.md, docs/ML_ENGINE_CONFIG.md, docs/ML_ENGINE_DESIGN.md, docs/ML_BACKEND_FRONTEND_INTEGRATION_PLAN.md, ml_config.py, ml_engine.py, training_worker.py, inference_worker.py.
- Admin Portal: AGENTS.md, docs/FRM_ADMIN_PORTAL_KB.md, src/pages, src/services, src/config, and profile/capability components.
- Read-only evidence baseline: docs/FRM_PLATFORM_PRD_BASELINE.md.

Appendix A - Deliverable Package

- docs/FRM_PLATFORM_PRD.md - authoritative source.
- docs/FRM_PLATFORM_PRD_CLIENT_APPENDIX.md - redacted client implementation profile.
- docs/FRM_PLATFORM_PRD_TRACEABILITY.md - requirement/source/UI/test matrix.
- docs/FRM_PLATFORM_PRD_DIAGRAMS.md - standalone diagrams and database lineage.
- docs/FRM_PLATFORM_PRD_BASELINE.md - read-only repository and database evidence baseline.
- docs/outputs/FRM_PLATFORM_PRD.docx - generated stakeholder document.
- docs/outputs/FRM_PLATFORM_PRD.pdf - generated distribution PDF.

Generated files must be regenerated when the authoritative Markdown is approved and changed.

Appendix A - Redacted Client Implementation

Source artifact: FRM_PLATFORM_PRD_CLIENT_APPENDIX.md

FRM Platform PRD - Redacted Client Implementation Appendix

Document ID: FRM-PRD-001-A **Version:** 1.0 **Status:** Draft for client/profile review **Prepared:** 2026-07-22

1. Purpose

This appendix explains how the common EFRM platform is shaped for an individual institution without embedding secrets, personal data, raw payloads, or live operational records in the platform PRD.

The appendix is a template for a client implementation workbook. Each client receives a completed, access-controlled copy with the actual approved values.

2. Client Profile Template

Field	Approved value
Institution identifier	[CLIENT_INSTITUTION_ID]
Database/tenant model	[Dedicated database / shared database with institution scope]
Active environment	[DEV / SIT / UAT / PROD]
Enabled services	[TRANSACTION, DEVICE, SCREENING, AMC, ML, RAMS, CASE, LINK]
Primary entity types	[CUSTOMER / INDIVIDUAL / ORGANIZATION / MERCHANT / OTHER]
Entity identity source	[Screening request / transaction payload / client master / other]
Case SOP owner	[Business owner]
Risk policy owner	[Risk owner]
Security policy owner	[Security owner]
External integration owner	[Client operations/technology owner]
Retention/legal-hold policy	[Approved policy reference]
UAT baseline	[Version/build/config bundle]

3. Service Enablement Matrix

Capability	Enabled?	Source system	Institution-specific configuration	Evidence required
Transaction fraud rules	[Y/N]	[Source]	Rule groups, metrics, thresholds, decision policy	Approved source contract and test cases
Device fraud rules	[Y/N]	[Source]	Device attributes, dimensions, decision policy	Device payload contract and test cases
Screening	[Y/N]	[Source]	List sources, entity forms, thresholds, bands, identifiers	Source licence/config and screening UAT
Adverse media	[Y/N]	[Provider]	Provider, categories, weights, country/recency, thresholds	Provider contract and test cases
ML inference	[Y/N]	[Source]	Feature/model versions, rollout, governance	Model manifest and approval
RAMS	[Y/N]	[All enabled sources]	Event admission, domains, weights, impact, decay, tiers	Risk policy approval
Case Management	[Y/N]	[All enabled alerts]	Case scoring, SLA, assignment, decisions/actions	SOP and closure UAT

Capability	Enabled?	Source system	Institution-specific configuration	Evidence required
Link Analysis	[Y/N]	[Transaction/device/entity sources]	Graph dimensions, lookback, limits, masking	Graph dimension approval

4. Client Configuration Bundle

A client configuration bundle should contain versioned, institution-scoped records for the following policy groups:

4.1 Case

- case_config_master;
- case_priority_master;
- alert_category_master;
- alert_count_boost_config;
- category_correlation_config;
- critical_override_rules where approved;
- sla_policy and sla_escalation;
- assignment_config;
- case_decision_master;
- case_action_master;
- case_decision_action_mapping;
- integration_endpoint_config.

The bundle must state which decisions require loss amount, recovery, remarks, evidence, or approval. It must also state which actions are mandatory, their order, target system, retry count, timeout, and success criteria. Endpoint credentials are references to the client's secret-management system, not values in the bundle.

4.2 RAMS

- risk_rating_config and effective version;
- risk_rating_weight_config;
- risk_rating_service_weight_config;
- risk_rating_tier_config;
- risk_rating_event_type_config;
- risk_rating_decision_impact_config;
- risk_rating_decay_policy_config;
- risk_rating_domain_override_config;
- risk_rating_relationship_risk_config;
- risk_rating_override_policy;
- entity_risk_config only after runtime-use confirmation.

The client must approve whether each enabled service participates in a risk domain. A transaction/device-only profile must not receive screening or AMC event admission merely because the table exists.

4.3 Screening

- source_config and source_column_mapping;
- list_source and source lifecycle;
- screening_config_master;
- screening_field_config;
- screening_group_config;
- screening_decision_threshold;
- screening_risk_band;

- screening_source_config;
- screening_identifier_config;
- screening_entity, screening_entity_relation, and screening_entity_field;
- whitelist_entry under a controlled exception workflow.

The client bundle must not contain source credentials or raw watchlist data. It may contain source IDs, mapping definitions, and approved policy references.

4.4 AMC

- adverse_config_master;
- adverse_integration_config without secrets;
- adverse_category_weight;
- adverse_severity_score;
- adverse_source_weight;
- adverse_country_risk;
- adverse_recency_factor;
- adverse_risk_band.

If AMC is disabled, the absence of an integration row may be intentional and must be recorded as such.

4.5 Rule Engine

- source_system_master;
- source_attribute_def;
- engine_attribute_def;
- engine_flink_map;
- rule_master and rule_version;
- rule groups, versions, maps, and source bindings;
- rule_decision_policy and rule_decision_upgrade;
- metric_definition and rule_metric_dependency;
- entity_definition and facts_definition where supported.

Rule versions and policy activation must be promoted with their dependencies. Derived required-data records should be generated by the rule workflow rather than hand-edited.

4.6 ML

- ml_feature_set;
- ml_feature_set_version;
- ml_feature_definition and metadata;
- ml_model_registry;
- ml_model_version;
- ml_model_governance;
- ml_score_blending_policy where supported.

Artifacts must be supplied through an approved artifact store and accompanied by a manifest containing checksums, feature version, training data profile, and approval.

4.7 Platform and Security

- institution and product/menu bootstrap;
- profile_master;
- profile_capability;
- password_policy_rule;
- security_check;

- notification_template;
- reference_data and enum values after API contract validation;
- service_client through secure provisioning;
- capability_endpoint_map through security-admin/migration workflow;
- efrm_service_config.

Institution, service-client, key, token, and secret records must be provisioned through deployment/security controls. They should not be copied into an unprotected SQL seed file.

5. Integration Contract Checklist

For every source or target system, record:

Item	Client value
System name	[Name]
Direction	[Inbound / outbound / bidirectional]
Transport	[REST / webhook / file / other]
Authentication	[Machine client / API key / mTLS / secret reference]
Timeout	[Approved value]
Retry policy	[Approved value]
Idempotency key	[Format and owner]
Success criteria	[HTTP/status/business response rules]
Payload owner	[Client/product team]
PII/secret fields	[Masking classification]
Failure owner	[Operations team]
Reconciliation method	[If applicable]
UAT evidence	[Test case/reference]

6. Client UAT Flow

6.1 Detection

- Submit a source payload with approved identity and request ID.
- Verify validation, persistence, rule/model evaluation, decision, score, and event lineage.
- Verify duplicate request behavior and failure response.
- Verify the alert appears in the correct queue and carries institution scope.

6.2 Investigation

- Open the alert and verify source details, signal/matches, risk context, and case linkage.
- Create a case or link to an existing case.
- Verify mixed alert mappings, assignment, SLA, timeline event, and audit event.
- Resolve one alert-level decision and verify the parent case remains open.

6.3 Closure

- Select the configured case outcome.
- Verify mandatory-field validation and approval gate.
- Preview mapped actions.
- Submit closure and verify case status, source alert statuses, unresolved/resolved mapping behavior, queue rows, audit/event rows, and RAMS case-decision event where enabled.

- Verify external action delivery/retry/manual resolution semantics.

6.4 Evidence and Compliance

- Upload evidence with an allowed type/size.
- Verify secure preview/download, version, SHA-256/hash, custody, and timeline/audit entries.
- Verify technical audit export includes only authorized filtered rows.
- Verify compliance readiness checks show explainable pass/fail reasons.

7. Client-Specific Acceptance Rules

A client deployment is accepted only when:

- the institution identifier is consistent in application, database, and seed/configuration bundle;
- enabled service configuration matches the approved scope;
- disabled service tables are not interpreted as enabled behavior;
- source IDs and timestamps are mapped correctly;
- action endpoints are configured with secret references and test success criteria;
- case outcome/action mappings match the approved SOP;
- RAMS event admission and decision impact match the approved risk policy;
- profile/menu/capability access matches least privilege;
- workers and migrations are deployed in the approved order;
- all UAT failures have an owner, severity, disposition, and retest evidence.

8. Redaction Rules

Do not include in a stakeholder copy:

- database passwords, JWT/private keys, API keys, provider secrets, raw auth headers, or secret hashes;
- customer names, identifiers, account/card/device/IP values, transaction payloads, or watchlist records;
- model artifact contents or unapproved training data;
- live endpoint credentials or unmasked action payloads;
- unrestricted SQL dumps or operational audit exports.

Use synthetic identifiers such as DEMO-CUSTOMER-001 and endpoint placeholders such as TARGET_SYSTEM_API.

9. Client Profile Sign-Off

Area	Owner	Approved?	Evidence
Enabled service scope	Product/client owner	[]	[]
Source payload contracts	Technology owner	[]	[]
Rule and metric policy	Rule owner	[]	[]
Screening/AMC policy	Compliance owner	[]	[]
RAMS policy	Risk owner	[]	[]
Case SOP/action policy	Operations/compliance owner	[]	[]
Portal access	Security owner	[]	[]
Retention/backup/recovery	Operations/security owner	[]	[]
UAT acceptance	Client approver	[]	[]

Appendix B - Requirements Traceability Matrix

Source artifact: FRM_PLATFORM_PRD_TRACEABILITY.md

FRM Platform PRD - Requirements Traceability Matrix

Document ID: FRM-PRD-001-T Version: 1.0 Prepared: 2026-07-22

How to Read This Matrix

The matrix links each requirement to the business objective, persona, implementation evidence, database/configuration dependency, portal coverage, acceptance criteria, failure/audit expectations, and test mapping. A requirement is not considered fully implemented merely because a table or route exists.

Status values:

- **IMPLEMENTED:** verified current behavior.
- **CONFIGURATION-DEPENDENT:** current code exists but the client must configure data/policy/integration.
- **PARTIAL:** part of the lifecycle or UI/API contract is incomplete.
- **PLANNED:** approved target work, not current capability.
- **OPEN:** requires a product/security/compliance/client decision.

Traceability Matrix

ID	Objective	Persona	Status	Repository/code evidence	DB/config evidence	Portal coverage	Acceptance and failure/audit expectation	Test mapping
FRM-PLT-001	Tenant isolation	Security admin	IMPLEMENTED	Admin Service auth/dependency and Institution access helpers	institution; institution-scoped tables	Protected routes and profile/capability UI	Cross-institution read/write denied and authorization failure audited where applicable	SEC-TENANT-001
FRM-PLT-002	Least privilege	Security admin	IMPLEMENTED	Profile/capability middleware and route authorization	profile_master; profile_capability; security_capability	Administration/profile screens	Hidden menu never replaces API authorization	SEC-RBAC-001
FRM-PLT-003	Credential/security policy	Security admin	CONFIGURATION-DEPENDENT	Auth middleware, password/session/security modules	password_policy_rule; security_check; service_client	Password/security screens	Effective policy visible; secrets never returned	SEC-AUTH-001
FRM-PLT-004	Service authentication	Operations	IMPLEMENTED	Service auth dependencies in each service	service_client and token/session records	No normal CRUD for secrets	Wrong/missing auth fails with stable response	SEC-SVC-001
FRM-PLT-005	Audit immutability	Auditor	IMPLEMENTED	Audit trail service and protected endpoints	audit_trail; security_audit_log	Audit/timeline read and export	No normal edit/delete; correction is append-only	AUD-IMM-001
FRM-PLT-006	Govern security policy	Security admin	PARTIAL	Capability/endpoint map support exists; restricted workflow incomplete	capability_endpoint_map; service_client	Profile/capability UI, no safe endpoint-map studio	High-impact changes require restricted authorization and audit	SEC-GOV-001
FRM-SCR-001	Watchlist lineage	Screening admin	IMPLEMENTED	ingestion_service; ingestion_worker; source/list APIs	source_config; source_column_mapping; list_source; list_version	Data Sources and ingestion screens	Version, source, mapping, entity/alias lineage retained	SCR-ING-001
FRM-SCR-002	Evidence-grounded matching	Analyst	IMPLEMENTED	fuzzy_screening_service and candidate repository	list_entity; list_entity_alias	Screening alert details	No list candidate means no invented match	SCR-MATCH-001
FRM-SCR-003	Configured screening policy	Screening admin	CONFIGURATION-DEPENDENT	screening_config_service and scoring	screening_config_master; field/group/threshold/band/source/id entifier tables	Screening config screens	Effective policy and validation visible; invalid policy rejects	SCR-CFG-001
FRM-SCR-004	Dynamic screening forms	Screening admin	IMPLEMENTED	screening_entity endpoints and validation	screening_entity; relation; field	Screening Entity/Form screens	Mandatory fields and relation payload remain compatible	SCR-FORM-001
FRM-SCR-005	Alert review	Analyst	IMPLEMENTED	screening alert endpoints and	screening_alert; screening_match;	Screening queue/details	Status/decision update is scoped	SCR-ALERT-001

ID	Objective	Persona	Status	Repository/code evidence	DB/config evidence	Portal coverage	Acceptance and failure/audit expectation	Test mapping
				audit hooks	screening_result		and audited	
FRM-SCR-006	Optional downstream integration	Product owner	CONFIGURATION-DEPENDENT	screening integration service paths	efrm_service_config; integration config	Limited/read-only in portal	Disabled downstream service creates no false row	SCR-INT-001
FRM-SCR-007	Exceptions/whitelist	Compliance officer	PARTIAL	whitelist repository/data path	whitelist_entry	No complete governed UI	Expiry, reason, approval, and audit required	SCR-WL-001
FRM-AMC-001	Async adverse request	Analyst	IMPLEMENTED	adversemedia API and request service	adverse_request	AMC request/review surface	Request ID/state returned before processing	AMC-REQ-001
FRM-AMC-002	Provider policy	AMC admin	CONFIGURATION-DEPENDENT	adverse runtime config/provider services	adverse_config_master; adverse_integration_config	AMC policy UI is partial	Effective DB/env/default source identified and secrets masked	AMC-CFG-001
FRM-AMC-003	State separation	Operations	IMPLEMENTED	AMC request/result/alert services	adverse_request; adverse_result; adverse_match; adverse_alert	AMC detail page	Provider failure distinct from result and alert	AMC-STATE-001
FRM-AMC-004	Explainable adverse score	Compliance officer	IMPLEMENTED	scoring/analysis services	category/severity/source/country/recency/band tables	Adverse detail tab	Contributing factors reviewable	AMC-SCORE-001
FRM-AMC-005	Optional RAMS/case path	Product owner	CONFIGURATION-DEPENDENT	integration callers	efrm_service_config; RAMS event config	Case/Risk operational surfaces	Client without AMC does not receive AMC event admission	AMC-INT-001
FRM-RUL-001	Canonical fact mapping	Rule admin	IMPLEMENTED	payload transform and attribute services	source_attribute_def; engine_attribute_def; engine_flink_map	Attribute/config screens	Source payload maps deterministically	RUL-MAP-001
FRM-RUL-002	Effective rule selection	Rule admin	IMPLEMENTED	RuleLoadServiceImpl and group binding services	rule_version; group/version/map/binding	Rule authoring/group screens	Only active effective binding runs	RUL-VERSION-001
FRM-RUL-003	Safe metric resolution	Rule admin	CONFIGURATION-DEPENDENT	metric loader/dependency/query services	metric_definition; rule_metric_dependency; entity_definition	Metric selector is partial	Limits and invalid dependency errors are visible	RUL-METRIC-001
FRM-RUL-004	Deterministic rule decision	Analyst	IMPLEMENTED	Drools execution, policy, upgrade services	rule_master; rule_version; decision policy/upgrade	Alert/result screens	Signal/decision/rule version persisted	RUL-EXEC-001
FRM-RUL-005	Source lineage	Operations	IMPLEMENTED	transaction/device orchestration and persistence	request/master/result/alert/match tables	Transaction/device details	Source/internal/result IDs remain distinct	RUL-ID-001
FRM-RUL-006	Failure isolation	Operations	PARTIAL	async aggregation/case/RAMS paths	queue/job/event tables	Error/refresh states	Downstream slow path does not block unrelated reads	RUL-RES-001
FRM-RUL-007	Rule governance	Rule admin	IMPLEMENTED	rule CRUD/version/activation APIs	rule_master; rule_version; audit records	Rule builder/admin	Activation actor/version/time recorded	RUL-GOV-001
FRM-ML-001	Feature contract	ML admin	IMPLEMENTED	ml_config and feature engineering services	ml_feature_set; ml_feature_set_version	Feature configuration pages	Active model references valid feature version	ML-FEAT-001
FRM-ML-002	Feature reproducibility	ML admin	IMPLEMENTED	feature_engineering_service	ml_feature_definition; metadata	Feature definition pages	Same version produces same vector contract	ML-FEAT-002
FRM-ML-003	Model lineage	ML governance	PARTIAL	model registry/version services	ml_model_registry; ml_model_version; governance	Model lifecycle partial	Artifact/metrics/training/version lineage complete	ML-MODEL-001
FRM-ML-004	Durable ML jobs	Operations	IMPLEMENTED	training_worker; inference_worker	training/inference/bulk job tables	Job status screens	API restart does not lose queued work	ML-WORKER-001
FRM-ML-005	Versioned inference	Analyst	IMPLEMENTED	ml_engine runtime and inference service	model version; feature version; inference result	Inference/simulation surfaces	Response identifies versions used	ML-INF-001
FRM-ML-006	Governance/rollback	ML governance	PARTIAL	governance/rollback APIs require contract review	ml_model_governance; ml_score_blending_policy	Incomplete/missing route coverage	Authorized, auditable lifecycle only	ML-GOV-001
FRM-ML-007	Downstream ML	Risk admin	CONFIGURATION-	ML output	risk event config	Risk/Case	Score/source/	ML-INT-001

ID	Objective	Persona	Status	Repository/code evidence	DB/config evidence	Portal coverage	Acceptance and failure/audit expectation	Test mapping
	event		DEPENDENT	integration	and inference records	operational UI	provenance preserved	
FRM-CASE-001	Case creation/merge	Analyst	IMPLEMENTED	case_management_service; case_scoring	case_master; case_alert_mapping	Case Queue/Detail	Open-case merge is institution/entity scoped and atomic	CASE-CREATE-001
FRM-CASE-002	Mixed alert case	Analyst	IMPLEMENTED	source identity and mapping logic	case_alert_mapping unique identity includes source table	Alerts tab	Same numeric IDs across source tables coexist	CASE-MIX-001
FRM-CASE-003	Assignment/SLA	Supervisor	CONFIGURATION-DEPENDENT	assignment/SLA services	assignment_config; sla_policy; sla_escalation	Assignment/SLA screens	Eligible user, SLA target, escalation visible	CASE-SLA-001
FRM-CASE-004	Alert-level decision	Analyst	IMPLEMENTED	case decision endpoints/service	case_alert_mapping decision fields	Case Alerts tab	Mapping resolves; parent case/actions unchanged	CASE-ALERT-DEC-001
FRM-CASE-005	Case outcome closure	Analyst/supervisor	IMPLEMENTED	case_api close; case_decision_service	case_master decision fields; decision config	Decision tab	Required decision/fields/approval enforced	CASE-CLOSE-001
FRM-CASE-006	Source closure consistency	Operations	IMPLEMENTED	close transaction and source update path	screening/adverse/transaction/device alert tables	Case alerts and source queues	All mapped source alerts close atomically; rollback on failure	CASE-SOURCE-001
FRM-CASE-007	Action initiation	Operations	IMPLEMENTED	case decision service and queue creation	case_action_master; mapping; endpoint; queue/log	Decision preview/action tracking	Queue rows contain config snapshot and idempotency	CASE-ACTION-001
FRM-CASE-008	Retry/manual resolution	Supervisor	IMPLEMENTED	case action worker and resolution endpoint	action queue/log manual fields	Action tracking panel	Retries and manual status are auditable	CASE-ACTION-002
FRM-CASE-009	Case risk signal	Risk admin	IMPLEMENTED	risk processing and case decision integration	risk_event_log; RAMS decision impact	Risk Profile/case summary	CASE/CASE_DECISION updates profile without rewriting source events	CASE-RISK-001
FRM-CASE-010	Evidence security	Investigator/auditor	PARTIAL	evidence API and secure preview/download paths	case_evidence; custody/event records	Evidence panel/gallery	Ownership/path/MIME/hash/custody are enforced	CASE-EVID-001
FRM-CASE-011	Audit/timeline separation	Auditor	IMPLEMENTED	case timeline/audit/compliance APIs	case_events; audit_trail; evidence/action/SLA records	Unified Timeline/Evidence workspace	Chronology and technical audit remain distinct	CASE-AUD-001
FRM-RAM-001	Event admission	Risk admin	IMPLEMENTED	risk ingestion/processing service	risk_rating_event_type_config	Risk operational screens	Unsupported events are ignored/rejected	RAMS-ADMIT-001
FRM-RAM-002	Weighted policy	Risk admin	CONFIGURATION-DEPENDENT	risk aggregation/processing	risk_rating_config and weights/impact	Risk display only; policy UI target	Effective policy and version visible	RAMS-CFG-001
FRM-RAM-003	Explainable profile	Analyst	IMPLEMENTED	aggregation/history/factor services	risk_event_log; aggregate/profile/factor/history	Risk Profile	Score and tier trace to contributing events/factors	RAMS-EXPLAIN-001
FRM-RAM-004	Decay/override/relationship	Risk admin	IMPLEMENTED	decay/override/relationship services	decay/override/relationship config	Policy UI target; operational read	Precedence and derived event are traceable	RAMS-POLICY-001
FRM-RAM-005	False-positive impact	Analyst/risk admin	IMPLEMENTED	case decision risk integration	decision impact config; risk event log	Decision/Risk Profile	Human signal suppressed/reduced; source events unchanged	RAMS-DEC-001
FRM-RAM-006	Versioned RAMS policy	Risk admin	PARTIAL	backend config APIs	risk_rating_config and child configs	No complete policy studio	Validation, approval, activation, diff	RAMS-GOV-001
FRM-LNK-001	Graph dimensions	Risk investigator	IMPLEMENTED	link_analysis_v2_service	graph_dimension_config/subtype	Operational graph only	Graph source lineage and dimensions visible	LINK-DIM-001
FRM-LNK-002	Graph refresh	Operations	IMPLEMENTED	link_analysis_refresh_worker	entity_graph_refresh_config/checkpoint	Refresh/status controls	Worker state durable and retryable	LINK-REFRESH-001
FRM-LNK-003	Graph safety	Risk administrator	CONFIGURATION-DEPENDENT	graph view/limit resolution	entity_graph_view_config	No governed config studio	Limits and lookback enforced	LINK-SAFETY-001
FRM-LNK-004	Cluster evidence	Investigator	IMPLEMENTED	graph cluster/evidence services	graph nodes/edges/evidence/clusters	V2 graph pages and case tab	Cluster/evidence traces to source	LINK-CLUSTER-001

ID	Objective	Persona	Status	Repository/code evidence	DB/config evidence	Portal coverage	Acceptance and failure/audit expectation	Test mapping
FRM-LNK-005	Govern graph config	Risk administrator	PLANNED	Target Link Analysis Configuration Studio	Graph policy tables	No current client self-service	Field/masking/relationship/limit validation	LINK-GOV-001
FRM-UI-001	Portal auth	All users	IMPLEMENTED	portal auth client, protected routes	user/session/token records	Login/ProtectedRoute	Invalid auth safely fails	UI-AUTH-001
FRM-UI-002	Portal capability visibility	Security admin	IMPLEMENTED	menuAccessResolver and capability components	menu_master; profile_capability	Sidebar/cards/profile mapping	Visibility and API auth agree	UI-ACCESS-001
FRM-UI-003	Effective scope	Admins	PARTIAL	config page patterns	institution/config root/version tables	Existing pages vary	High-impact pages show institution/version/effective state	UI-SCOPE-001
FRM-UI-004	Runtime vs config	All users	IMPLEMENTED	page/service separation	runtime vs config table classes	Multiple domains	No generic runtime CRUD	UI-DATA-001
FRM-UI-005	Safe export	Auditor	PARTIAL	shared export utilities and audit page	audit/case/evidence data	CSV/Excel varies by page	Filtered authorized export works and is auditable	UI-EXPORT-001
FRM-OPS-001	Worker operations	Operations	IMPLEMENTED	case action/link refresh/training/inference workers	queue/job/checkpoint tables	Status/refresh surfaces	Restart-safe durable jobs	OPS-WORKER-001
FRM-OPS-002	Deployment secrets	Operations/security	IMPLEMENTED	env/config/deployment manifests	service client/config refs	Sanitized diagnostics only	Raw secrets never in client UI or PRD	OPS-SECRET-001

Database Coverage References

The authoritative configuration table inventory and UI coverage classifications are maintained in:

- docs/PRODUCT_CONFIGURATION_UI_COVERAGE_AUDIT.md
- docs/PRODUCT_CONFIGURATION_UI_COVERAGE_DETAILED_AUDIT.md

Those reports explicitly classify uncertain/legacy candidates as REVIEW REQUIRED and distinguish configuration from runtime, derived, audit/history, and worker state.

Release Traceability Gate

Before stakeholder sign-off, each IMPLEMENTED row must have:

1. an API or UI test evidence reference;
2. a database/configuration validation query or migration check;
3. a failure-path result;
4. an audit/event expectation;
5. a client-profile decision where behavior is configuration-dependent.

Appendix C - Architecture and Data-Lineage Diagrams

Source artifact: FRM_PLATFORM_PRD_DIAGRAMS.md

FRM Platform PRD - Architecture, Sequence and Data-Lineage Diagrams

Document status: Supporting source artifact for the Enterprise FRM Platform PRD **Evidence posture:** Current-state diagrams are based on repository documentation, service entrypoints, API modules, models, migrations, worker scripts, and the configuration inventory. Where behavior depends on client configuration or an external system, the dependency is shown explicitly. **Related documents:** FRM_PLATFORM_PRD.md, FRM_PLATFORM_PRD_CLIENT_APPENDIX.md, FRM_PLATFORM_PRD_TRACEABILITY.md, and FRM_PLATFORM_PRD_BASELINE.md

1. Logical Architecture

MERMAID DIAGRAM / REFERENCE BLOCK

```

flowchart LR
    U[Analyst / Investigator / Admin]
    P[Admin Portal]
    G[Authenticated API boundary]
    A[Admin Service\nCase / RAMS / Link / Audit / Config]
    S[Screening Service\nWatchlist / Alerts / Ingestion]
    M[AMC Service\nAdverse Media / Providers]
    R[Rule Engine\nTransaction / Device rules]
    L[ML Engine\nFeatures / Models / Inference]
    DB[(PostgreSQL\nefrm schema)]
    W[Workers / Schedulers]
    X[Client systems\nCBS / Card / Switch / CRM / Notification]
    V[Watchlist and media sources]

    U --> P --> G
    G --> A
    G --> S
    G --> M
    G --> R
    G --> L
    S --> DB
    M --> DB
    R --> DB
    L --> DB
    A --> DB
    W --> DB
    W --> X
    S --> V
    M --> V
    R -. alert/result contract .-> A
    S -. alert/result contract .-> A
    M -. alert/result contract .-> A
    L -. score / explainability .-> A
  
```

2. Request and Decision Flow

The platform has two distinct decision boundaries:

- Detection decision:** rule, screening, AMC, or ML processing produces a result/alert and an initial machine decision or score.
- Investigation decision:** an investigator selects an alert-level resolution or a final case outcome. Only the final case outcome initiates configured case actions.

MERMAID DIAGRAM / REFERENCE BLOCK

```
sequenceDiagram
    autonumber
    participant C as Client Source
    participant R as Rule/Screening/AMC Service
    participant D as Admin Service
    participant DB as PostgreSQL
    participant P as Admin Portal
    participant W as Action Worker
    participant E as External Target System

    C->>R: Submit transaction, device, screening, or AMC request
    R->>DB: Persist request, result, score, and alert
    R->>D: Create or merge case from alert context
    D->>DB: Persist case_master and case_alert_mapping
    P->>D: Load case, alerts, decisions, and action preview
    D-->>P: Investigation workspace
    P->>D: Submit alert-level decision (optional)
    D->>DB: Close mapping and write decision event
    Note over D,DB: No SOP action and no parent-case closure
    P->>D: Submit final case decision
    D->>DB: Validate SOP, persist case decision, close unresolved mappings
    D->>DB: Queue mapped action rows and write audit/event records
    D-->>P: Case closed from RiskBrains side
    W->>DB: Claim pending action snapshot
    W->>E: Dispatch REST/webhook/internal action
    E-->>W: Delivery response or failure
    W->>DB: Persist delivery status, retry metadata, and immutable log
```

3. Transaction and Device Intake

MERMAID DIAGRAM / REFERENCE BLOCK

```
sequenceDiagram
    autonumber
    participant I as Client / GUI Intake
    participant R as Rule Engine
    participant CFG as Rule Configuration
    participant DB as PostgreSQL
    participant A as Admin Service
    participant K as Case Service

    I->>R: Submit source payload with institution and source transaction key
    R->>CFG: Resolve source, fact, attributes, metrics, rule groups, policy
    CFG-->>R: Active versioned configuration
    R->>R: Transform payload into engine facts
    R->>R: Execute rules and resolve upgrades/policy
    R->>DB: Persist request/master/result/match rows
    R->>A: Publish or expose result/alert context
    A->>K: Evaluate case-creation/merge policy
    K->>DB: Insert case or map alert to existing open case
    K->>DB: Keep alert source identity and case aggregate current
```

Identity note: the source transaction identifier is client-owned (source_txn_id in the current contract). Internal transaction/request identifiers are platform identifiers and must not be substituted when correlating external events.

4. Screening Flow

MERMAID DIAGRAM / REFERENCE BLOCK

```
sequenceDiagram
    autonumber
    participant C as Client / Portal
    participant S as Screening Service
    participant CFG as Screening Configuration
    participant DB as PostgreSQL
    participant A as AMC / Admin Service
    participant W as Screening Worker

    C->>S: Submit entity screening request
    S->>CFG: Resolve field, identifier, source, threshold, risk-band config
    S->>DB: Read active list entities, aliases, relations, and identifiers
    S->>S: Normalize request and retrieve candidate records
    S->>S: Score exact, identifier, DOB, country, and fuzzy evidence
    S->>DB: Persist screening_request, screening_result, and matches
    S->>DB: Persist screening alert when configured threshold is met
    S->>A: Optional AMC trigger for configured screening context
    A->>DB: Persist adverse request/results and media matches
    S->>DB: Persist consolidated alert/risk integration references where enabled
    S->>A: Case integration for alert creation or merge
    W->>DB: Ingest or process source-list batches asynchronously
```

5. AMC / Adverse Media Flow

MERMAID DIAGRAM / REFERENCE BLOCK

```
sequenceDiagram
    autonumber
    participant C as Client / Screening Service
    participant M as AMC Service
    participant CFG as AMC Configuration
    participant P as Search Providers
    participant DB as PostgreSQL
    participant A as Admin Service

    C->>M: Submit adverse media request
    M->>CFG: Resolve provider, source, category, severity, country, recency config
    M->>DB: Persist adverse_request
    M->>P: Execute provider/search request
    P-->>M: Search results and source documents
    M->>M: Extract, normalize, deduplicate, classify, and score
    M->>DB: Persist adverse_result and adverse_match rows
    M->>DB: Persist adverse alert when configured
    M->>A: Optional RAMS and case integration
    A->>DB: Add adverse alert to case or create/merge a case
    Note over M,DB: Entity and timestamp backfill uses adverse_request when not stored on adverse_result
```

6. ML Configuration and Runtime

MERMAID DIAGRAM / REFERENCE BLOCK

```
sequenceDiagram
    autonumber
    participant O as ML/Admin User
    participant P as Admin Portal
    participant M as ML Service
    participant DB as PostgreSQL
    participant W as ML Worker
    participant D as Detection / Case Consumers

    O->>P: Define feature set and model metadata
    P->>M: Create or update configuration
    M->>DB: Persist feature set, definitions, versions, registry, governance
    W->>DB: Claim training or inference job
    W->>W: Build features and execute approved model version
    W->>DB: Persist score, explanation, feedback, and run status
    M->>D: Expose score/inference result
    D->>DB: Use score in alert, case, or downstream risk context
```

7. Case Lifecycle and Alert Mapping

MERMAID DIAGRAM / REFERENCE BLOCK

```
stateDiagram-v2
    [*] --> OPEN: eligible alert/case creation
    OPEN --> IN_PROGRESS: assignment or investigation begins
    IN_PROGRESS --> DECISION_SUBMITTED: valid final decision submitted
    DECISION_SUBMITTED --> PENDING_APPROVAL: decision requires approval
    DECISION_SUBMITTED --> ACTION_PENDING: no approval gate
    PENDING_APPROVAL --> ACTION_PENDING: approved
    PENDING_APPROVAL --> IN_PROGRESS: rejected
    ACTION_PENDING --> ACTION_DISPATCHED: mapped actions queued/dispatch initiated
    ACTION_DISPATCHED --> CLOSED: RiskBrains-side closure committed
    ACTION_PENDING --> ACTION_FAILED: dispatch failure recorded
    ACTION_FAILED --> ACTION_DISPATCHED: retry succeeds or manual resolution
    ACTION_FAILED --> CLOSED: permitted manual resolution / V1 policy
```

MERMAID DIAGRAM / REFERENCE BLOCK

```
flowchart TD
    C[case_master] --> M[case_alert_mapping]
    M --> SA[screening_alert]
    M --> AA[adverse_alert]
    M --> TA[transaction_alert]
    M --> DA[device_alert]
    C --> D[case decision fields]
    D --> DM[case_decision_master]
    DM --> MAP[case_decision_action_mapping]
    MAP --> AM[case_action_master]
    AM --> EP[integration_endpoint_config]
    D --> Q[case_action_execution_queue]
    Q --> L[case_action_execution_log]
    C --> CE[case_events]
    C --> AT[audit_trail]
    C --> EV[case_evidence]
```

Alert-level rule: case_alert_mapping is the source of truth for an alert's investigation-level resolution. Its alert_source_table remains part of identity, so screening and adverse rows with the same numeric ID cannot be confused. Alert-level decisions do not queue actions and do not close case_master.

8. RAMS / Risk Rating Lineage

MERMAID DIAGRAM / REFERENCE BLOCK

```

flowchart LR
    T[transaction_result]
    DV[device_result]
    SC[screening_result]
    AM[adverse_result]
    CD[case decision]
    E[risk_event_log]
    IMP[risk_rating_decision_impact_config]
    EVT[risk_rating_event_type_config]
    W[risk_rating_weight_config]
    SW[risk_rating_service_weight_config]
    DEC[risk_rating_decay_policy_config]
    OVR[risk_rating_override_policy]
    AGG[entity_risk_aggregate]
    PROF[entity_risk_profile]
    HIST[entity_risk_history]

    T --> E
    DV --> E
    SC --> E
    AM --> E
    CD --> E
    IMP --> E
    EVT --> E
    E --> W
    E --> SW
    E --> DEC
    E --> OVR
    E --> AGG
    AGG --> PROF
    PROF --> HIST
  
```

Processing rule: inserting a row in risk_event_log is not by itself a complete risk-rating operation. Events that should affect the entity profile/aggregate must pass through the existing risk processing path. Case decisions use the established source_type=CASE and event_type=CASE_DECISION convention when configured to update risk.

9. Link Analysis Configuration and Refresh

MERMAID DIAGRAM / REFERENCE BLOCK

```

sequenceDiagram
    autonumber
    participant O as Admin / Operations
    participant A as Admin Service
    participant CFG as Graph Configuration
    participant DB as PostgreSQL
    participant W as Refresh Worker
    participant P as Portal

    O->>A: Configure graph dimensions, views, refresh policy, and checkpoints
    A->>DB: Persist graph_dimension_config and related tables
    W->>DB: Claim due refresh/checkpoint
    W->>DB: Read transaction/device/entity source data
    W->>W: Build nodes, edges, evidence, and risk clusters
    W->>DB: Persist graph snapshot, cluster, evidence, and checkpoint status
    P->>A: Request V2 clusters or case graph
    A->>DB: Read latest permitted graph snapshot
    A-->>P: Cluster metadata, graph nodes, edges, and evidence
  
```

10. Audit, Timeline, Evidence and Compliance Lineage

MERMAID DIAGRAM / REFERENCE BLOCK


```

flowchart LR
    OP[Case operation]
    CE[case_events\nBusiness timeline]
    AT[audit_trail\nTechnical audit]
    EV[case_evidence\nFile metadata/version/hash]
    CUST[Evidence custody\nJSON/event history]
    SLA[SLA trackers]
    DEC[Case decision/approval]
    ACT[Action queue/log]
    API[Case-specific read APIs]
    UI[Unified Timeline & Evidence workspace]

    OP --> CE
    OP --> AT
    OP --> EV
    EV --> CUST
    DEC --> SLA
    DEC --> ACT
    CE --> API
    AT --> API
    EV --> API
    CUST --> API
    SLA --> API
    ACT --> API
    API --> UI

```

Interpretation boundary: Timeline answers “what happened in the business investigation?” Technical Audit answers “what technical operation was recorded, by whom, when, from which source, and with what stored changes?” Evidence/Custody answers “what supporting file/version was provided, viewed, downloaded, hashed, and retained?” Compliance readiness evaluates the available correlated records; it does not reconstruct events that were never persisted.

11. Configuration-to-UI Coverage Model

MERMAID DIAGRAM / REFERENCE BLOCK

```

flowchart TD
    DB[Configuration table]
    API[Admin/config API]
    UI[Portal configuration screen]
    SEED[Seed/migration/defaults]
    RUNTIME[Runtime consumer]
    REVIEW[REVIEW REQUIRED]

    DB --> API
    API --> UI
    DB --> SEED
    DB --> RUNTIME
    DB -. no verified UI mapping .-> REVIEW

```

Every configuration table should be assessed against three independent questions:

Question	Meaning	PRD treatment
Is it persisted?	The table and lifecycle exist in the schema/model/migrations.	Document owner, keys, lifecycle, and client scope.
Is it consumed?	A runtime path reads it and applies the configuration.	Document process, precedence, cache, and failure behavior.
Is it operable?	An authorized portal/API flow can safely change it.	Mark IMPLEMENTED, PARTIAL, or OPEN; do not imply UI support from DB presence.

12. Table Classification Summary

The detailed configuration audit identifies 94 configuration/governance candidates across the platform. The following classification is the PRD baseline; exact field-level and UI coverage remains in PRODUCT_CONFIGURATION_UI_COVERAGE_DETAILED_AUDIT.md and the main PRD.

Domain	Configuration/governance candidates	Main runtime/derived/audit families	Primary owner
Platform/security	17	Requests, sessions, audit, notifications	Admin Service / Portal

Domain	Configuration/governance candidates	Main runtime/derived/audit families	Primary owner
Case management	13	Cases, mappings, events, evidence, actions, SLA state	Admin Service
RAMS	11	Risk events, aggregate, profile, history	Admin Service
Link Analysis	4	Graph snapshots, nodes, edges, clusters, checkpoints	Admin Service
Rule Engine	19	Requests, rule executions, matches, decisions, results	Rule Engine
Screening	14	Screening requests/results, list entities, matches, alerts	Screening Service
AMC	8	Adverse requests/results/matches/alerts	AMC Service
ML	8	Training/inference runs, predictions, explanations, feedback	ML Service
Total	94	Runtime and history are classified separately	Cross-service

13. Deployment and Trust Boundaries

MERMAID DIAGRAM / REFERENCE BLOCK

```

flowchart TB
    subgraph ClientZone[Client / Institution Zone]
        CBS[Core Banking]
        CARD[Card / Payment Systems]
        CRM[CRM / Notification / Recovery]
    end
    subgraph PlatformZone[FRM Platform Zone]
        PORTAL[Admin Portal]
        APIS[Service APIs]
        WORKERS[Workers and schedulers]
        PG[(PostgreSQL / efrm)]
        SECRETS[Secret/profile injection]
    end
    subgraph SourceZone[External Data Sources]
        LISTS[Watchlists]
        MEDIA[Media/search providers]
    end

    PORTAL --> APIS
    APIS --> PG
    WORKERS --> PG
    WORKERS --> CBS
    WORKERS --> CARD
    WORKERS --> CRM
    APIS --> LISTS
    APIS --> MEDIA
    SECRETS --> APIS
    SECRETS --> WORKERS

```

The PRD treats institution identifiers, database URLs, credentials, tokens, provider keys, and payload secrets as deployment inputs. They are intentionally absent from this document and from the redacted client appendix.

14. Evidence and Review Status

Evidence source	Use in PRD	Limitation / treatment
Repository source, routes, models, services	Functional ownership and API behavior	Runtime behavior can still vary by environment/configuration.
Alembic migrations and SQL scripts	Schema lifecycle and compatibility	A migration file is not proof that every environment has applied it.
Existing configuration coverage audit	Configuration inventory and UI coverage	Uncertain candidates remain explicitly REVIEW REQUIRED.
PostgreSQL client DBs	Client-profile and schema verification	No live credentials or operational rows are included in deliverables.

Evidence source	Use in PRD	Limitation / treatment
Portal routes, permissions, API clients	UI coverage and authorization	UI availability does not imply backend permission or client data readiness.
Tests and smoke checks	Acceptance mapping	Passing tests do not replace UAT evidence for external integrations.

Appendix D - Read-Only Evidence Baseline

Source artifact: FRM_PLATFORM_PRD_BASELINE.md

FRM Platform PRD - Read-Only Evidence Baseline

Document ID: FRM-PRD-001-B **Version:** 1.0 **Captured:** 2026-07-22 **Purpose:** Record the repository baseline used for the platform PRD. This is evidence metadata, not a release manifest.

1. Repository Baseline

Product area	Repository	Branch at capture	HEAD commit at capture	Remote state observed	Working-tree note
Admin Service	ffps-efrminservice	main	95d8deef0d658c39a51aee31c564a265aa624fac	Local main aligned with origin/main at capture	Pre-existing local/editor/demo artifacts present; not changed for PRD.
Screening Service	ffps-efrmscreeningservice	main	495e362e0ad5f9c8f34624d1f8aa90b18e54acdb	Local main aligned with origin/main at capture	Pre-existing environment/bytecode artifacts present; not changed for PRD.
AMC Service	ffps-efrmamcservice	main	e00e64b35e146bb46507f3bf96d4a670a5472b79	Local main aligned with origin/main at capture	Pre-existing documentation/untracked artifacts present; not changed for PRD.
Rule Engine Service	ffps-efrmruleengineservice	main	e97d73f820ec6b927b5abdd9106114f7d3db5369	Local main aligned with origin/main at capture	Pre-existing documentation/scripts/dependency artifacts present; not changed for PRD.
ML Engine Service	ffps-efrmmlengineservice	main	fc72e2fd9b7fb853a4f2f395ae4e295df0c5a9e	Local main aligned with origin/main at capture	Pre-existing repository guidance/documentation artifacts present; not changed for PRD.
Admin Portal	ffps-frm-admin-portal	main	c04bf7dab24fa249c9ae53b575c0c0cfcea18abd	Local main aligned with origin/main at capture	Pre-existing repository guidance/documentation/assets present; not changed for PRD.

2. Database Evidence Boundary

The discovery review considered the local PostgreSQL client databases Kanji_011, efrm_niyo, and efrm_uat_latest using the efrm application schema. The PRD uses these databases to validate schema/configuration vocabulary and client-profile differences; it does not reproduce their operational rows or credentials.

Database facts in this appendix are intentionally limited to:

- schema/table presence and model/migration alignment;
- institution-scoped configuration concepts;
- service enablement and client-profile differences;
- runtime/configuration/audit/worker table classification.

3. Review Method

The baseline was assembled read-only from:

1. repository branch/commit/remote/worktree metadata;
2. API route and schema definitions;

3. service, worker, scheduler, and integration modules;
4. SQLAlchemy/JPA models, migrations, and seed/configuration scripts;
5. existing tests and repository documentation;
6. portal pages, API clients, profile/capability resolution, and menu mappings;
7. PostgreSQL schema and configuration checks for the named client databases.

No source code, database schema, database row, configuration value, service process, or runtime behavior was changed while preparing the PRD.

4. Baseline Limitations

- A repository baseline records what was inspected; it is not proof that a deployed environment runs the same commit.
- A migration file is not proof that every database has applied that migration.
- A configuration row is not proof that a capability is enabled for every institution.
- A passing unit test is not proof that an external client integration has completed UAT.
- Any table or behavior not confidently classified remains REVIEW REQUIRED in the configuration audit and traceability matrix.